Universidad Autónoma de Tamaulipas

Facultad de Comercio, Administración y Ciencias Sociales

Nuevo Laredo

Actividad 6. Faker

Análisis de ventas con Pandas y Faker

Carlos Gabriel Rodríguez Macías

Licenciatura en Tecnologías de la Información

Temas Selectos de Tratamiento de Información

Athala Dulce Edith Gómez Arellano

16 de octubre de 2026

Análisis, filtrado y generación de datos

Una cadena de tiendas desea analizar las ventas realizadas durante el último año para identificar los productos con mayor demanda y generar reportes para la toma de decisiones.

Preguntas de reflexión

1. ¿Qué ventajas ofrece renombrar las columnas de un DataFrame antes de comenzar el análisis de datos?

Renombrar las columnas mejora la claridad y permite identificar con facilidad qué representa cada campo. También ayuda a mantener nombres consistentes, reduce errores al escribir referencias a las columnas y hace más comprensibles los análisis, las tablas y las visualizaciones.

2. ¿En qué situaciones es útil filtrar los datos de un DataFrame en lugar de trabajar con todos los registros?

Filtrar es útil cuando se necesita analizar solamente los registros que cumplen una condición, como productos con precio mayor a 500, ventas de un periodo específico o datos sin valores nulos. De esta manera se trabaja con información relevante y se obtienen resultados más precisos para el objetivo del análisis.

Evidencia del programa

Figura 1

Código completo para el análisis de ventas y la generación de registros con Pandas y Faker



```python
# Actividad 6
# ? Athala Dulce Edith Gómez Arellano
# ! Carlos Gabriel Rodriguez Macias
# * 16 de Octubre de 2026

# ! Problema:
# * Una cadena de tiendas desea analizar las ventas realizadas durante el último año para
# * identificar los productos con mayor demanda y generar reportes para la toma de decisiones.

# ! Instrucciones
# ? 1 Importa la biblioteca pandas y Faker. Configura Faker para generar datos de México (es_MX).
import pandas as pd
from pathlib import Path
from faker import Faker

fake = Faker('es_MX')

# ? 2 Empleando el control de errores, crea una variable llamada datos_csv y carga el
# ? archivo CSV, agrega un mensaje en caso de que no exista el archivo y utiliza else para
# ? mostrar la información general del DataFrame.
try:
    ruta_csv = Path(__file__).with_name("Datos_Ventas_Tienda.csv")
    datos_csv = pd.read_csv(ruta_csv, encoding="utf-8-sig")
except FileNotFoundError as e:
    print(f"El archivo no existe: {e}")
else:
    print("Información general del DataFrame:")
    print(datos_csv.info())

# ? 3 Convierte la columna Fecha del archivo CSV al formato día-mes-año
Resultado = pd.to_datetime(datos_csv["Fecha"],errors="coerce", format="%m/%d/%Y").dt.strftime("%d/%m/%Y")

# ? 4 Filtra e imprime los registros del archivo CSV donde el Precio Unitario >500
filtrado_precio = datos_csv[datos_csv["Precio Unitario"] > 500]

# ? 5 Investiga el método pydecimal()
# * El método pydecimal() se utiliza para crear objetos de tipo Decimal en Python, que permiten
# * realizar cálculos con mayor precisión que los tipos de datos flotantes. Esto es especialmente útil
# * en aplicaciones financieras y científicas donde la exactitud es crucial.

# ? 6 Genera un número entero aleatorio entre 1 y 100, empleando Faker
print(f"Número aleatorio entre 1 y 100: {fake.random_int(min=1, max=100)}")

# ? 7 Genera una lista llamada materias que contenga 5 nombres de asignaturas e imprime 2
# ? valores que puedan repetirse.
materias = ["Base de datos", "Programación", "Matemáticas", "Estadística", "Inteligencia Artificial"]
asignaturas_random = fake.random_elements(elements=materias, length=2, unique=False)
print(f"Asignaturas seleccionadas: {asignaturas_random}")

# ? 8 Renombra la columna Precio Unitario a Precio e imprime los primeros 5 registros
datos_csv.rename(columns={"Precio Unitario": "Precio"}, inplace=True)
print("Primeros 5 registros con la columna renombrada:")
print(datos_csv.head(5))

# ? 9 Convierte los valores de la columna Fecha en el formato dd/mm/yyyy
fechas = pd.to_datetime(
    datos_csv["Fecha"], errors="coerce", format="%m/%d/%Y"
).dt.strftime("%d/%m/%Y")
print("Fechas convertidas al formato dd/mm/yyyy:")
print(fechas)




# ? 10 Crea una lista vacía llamada datos_exportar
datos_exportar = []

# ? 11 Emplea un ciclo for para generar 15 registros, utilizando diccionarios que deberán contener
# ? lo siguiente y emplea el método append() para agregar los datos
for i in range(15):
    datos_exportar.append({
        "Nombre": fake.first_name(),
        "Apellido": fake.last_name(),
        "Número Aleatorio": fake.random_int(min=1, max=100),
        "Materia": fake.random_element(elements=materias),
        "Fecha": fechas.iloc[i],
        "Total Venta": datos_csv["Total Venta"].iloc[i]
    })

# ? 12 Muestra los registros almacenados en la lista
print("Registros almacenados en la lista datos_exportar:")
for registro in datos_exportar:
    print(registro)
```